

NPTEL

NPTEL Course on
"Computer System Optimization for ML"

Lecture #19

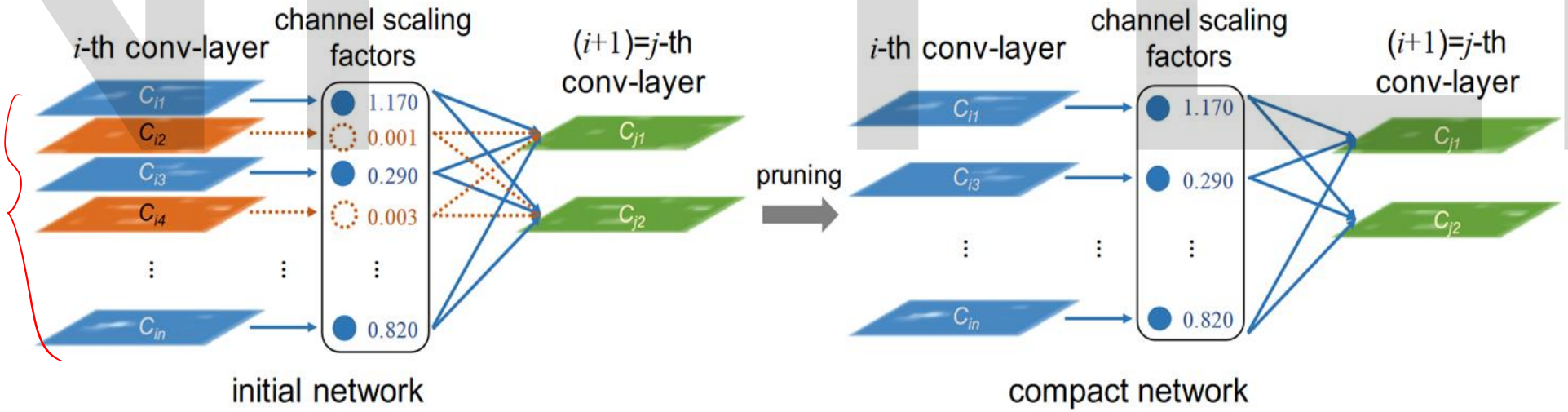
Agenda

- Recap
- DNN Pruning (contd.)
- Parallelism in DNN Training

NPTEL

Which Parameters to Prune: Scaling-based Pruning

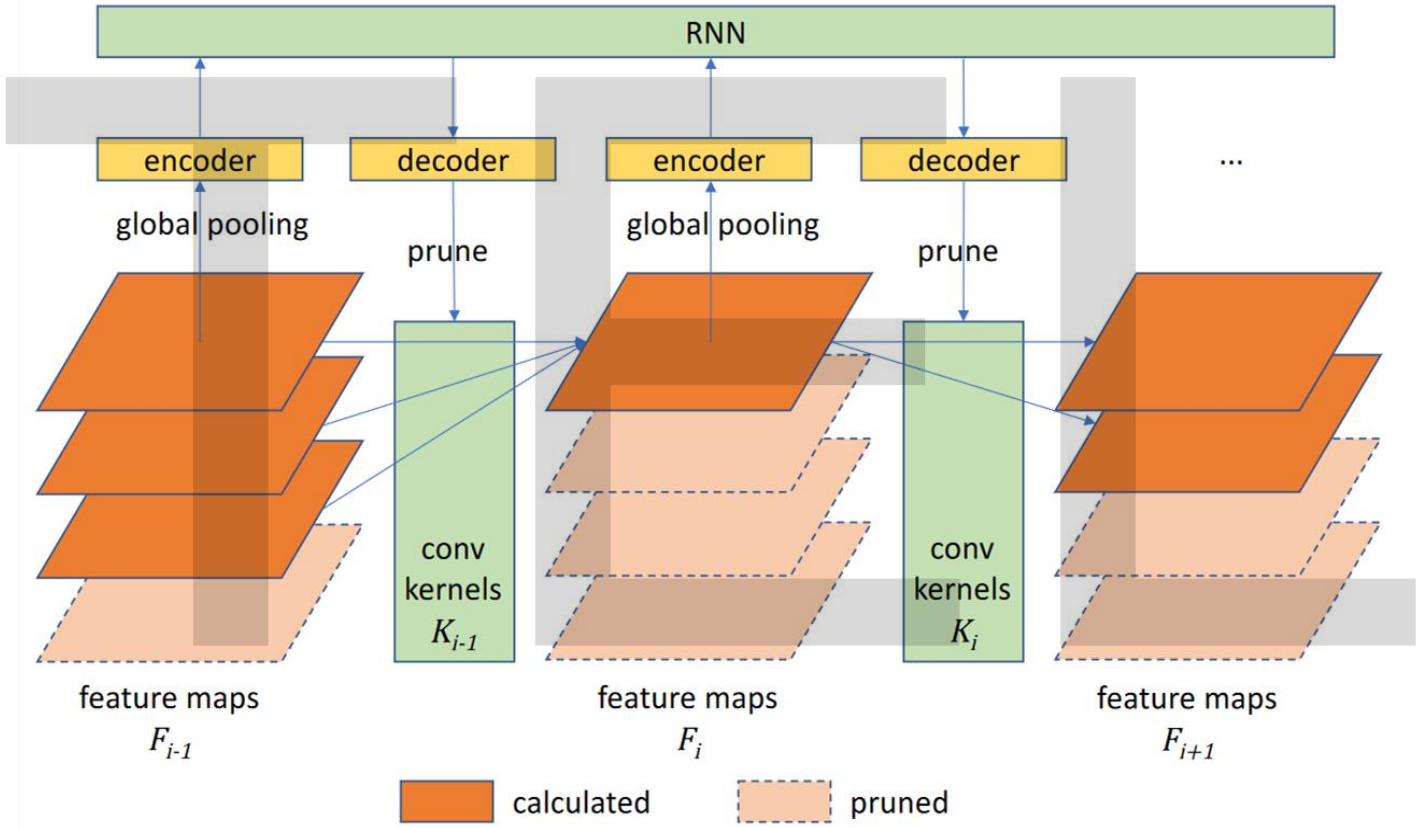
- The aim of scaling-based pruning is to find the important and redundant layers while training the network itself
- For channel pruning of CNN, each channel can be associated with a scaling factor which are trainable parameters
- The channels with low scaling factors can be removed



Ref: Learning Efficient Convolutional Networks through Network Slimming [Liu et al., ICCV 2017]

Dynamic Pruning

- **Dynamic Pruning or Runtime Pruning can consist of 2 sub-networks**
  - CNN Backbone
  - Decision Network
- **A mask variable can be used to denote the filters that will be used in the forward pass for the current iteration**



Ref: Pruning and Quantization for Deep Neural Network Acceleration: A Survey, Liang et al.

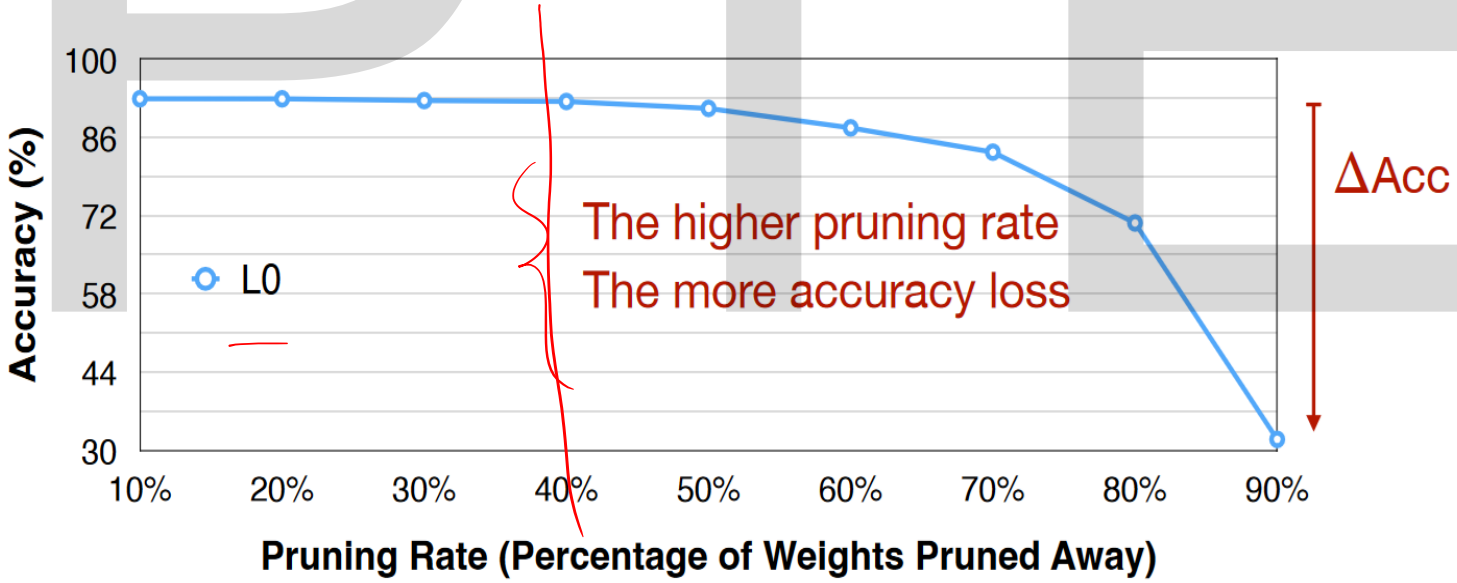
Agenda

- Recap
- **DNN Pruning (contd.)**
- Parallelism in DNN Training

NPTEL

Pruning in Different Layers

- A deep neural network consists of many layers which hypothetically specializes at identifying different level of patterns in the data
- How do different layers react to pruning? Can we prune different layers with the same pruning ratio or are there some layers which are more sensitive to pruning compared to others?

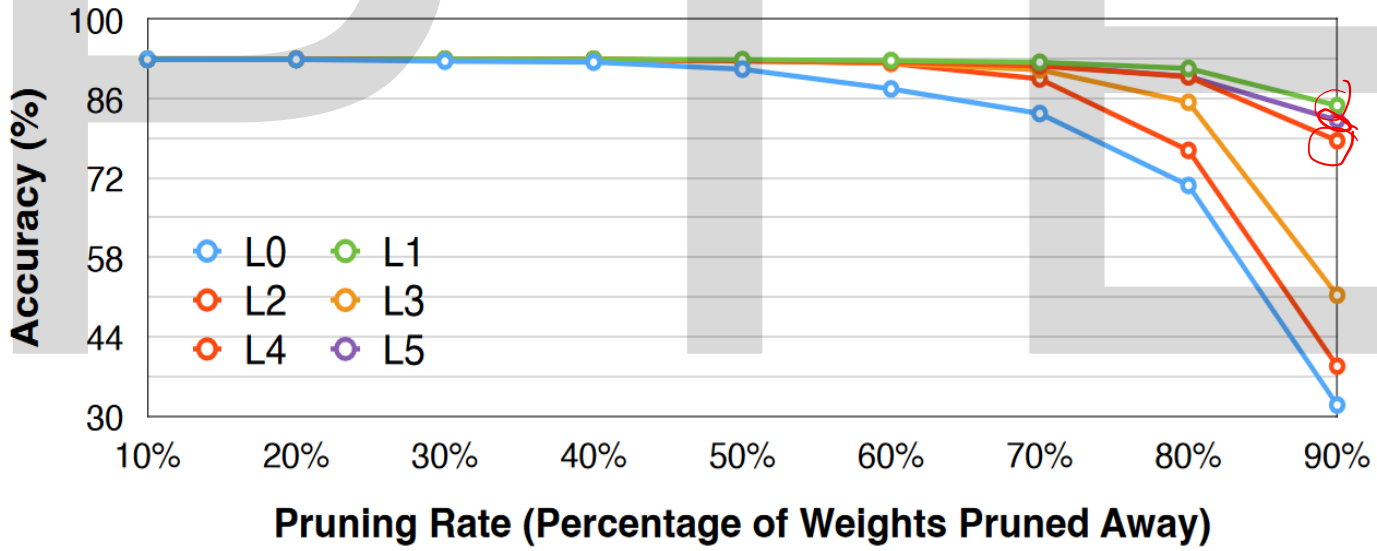


Pruning different layers of VGG on CIFAR 10

NPTEL

Pruning in Different Layers

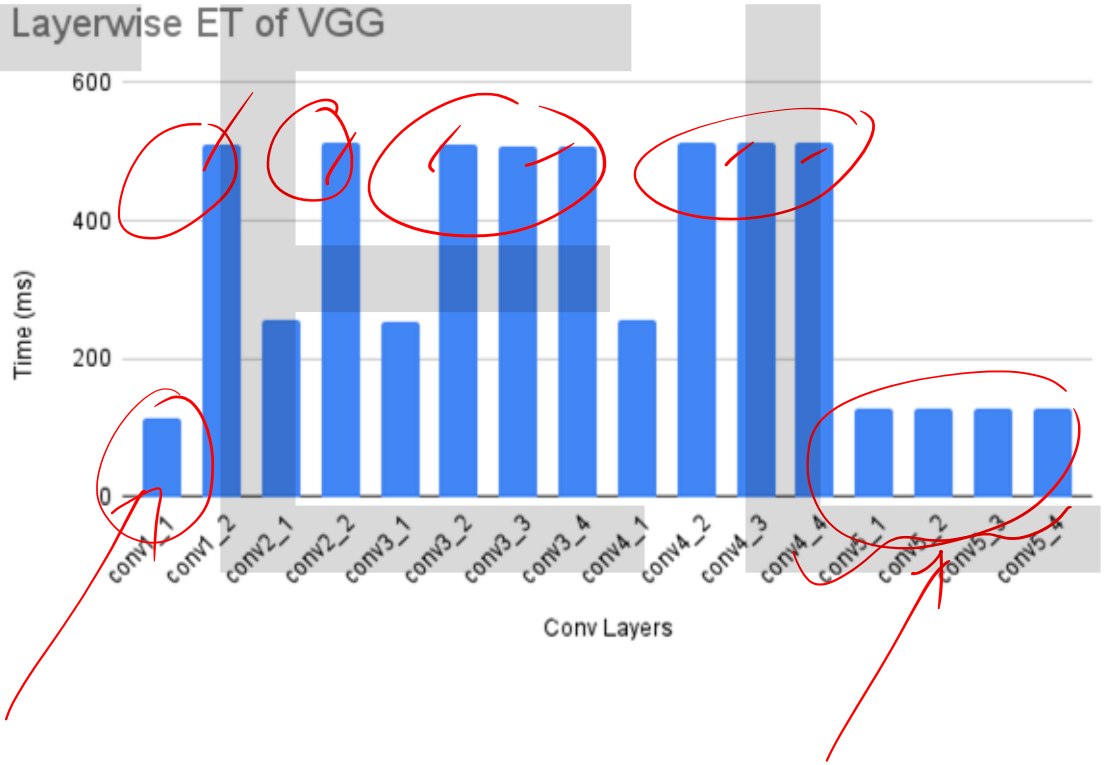
- We see that some layers are more sensitive to pruning than others
- Typically, layers deeper in the network can be pruned more without much loss in accuracy compared to the initial layers



NPTEL

Pruning in Different Layers

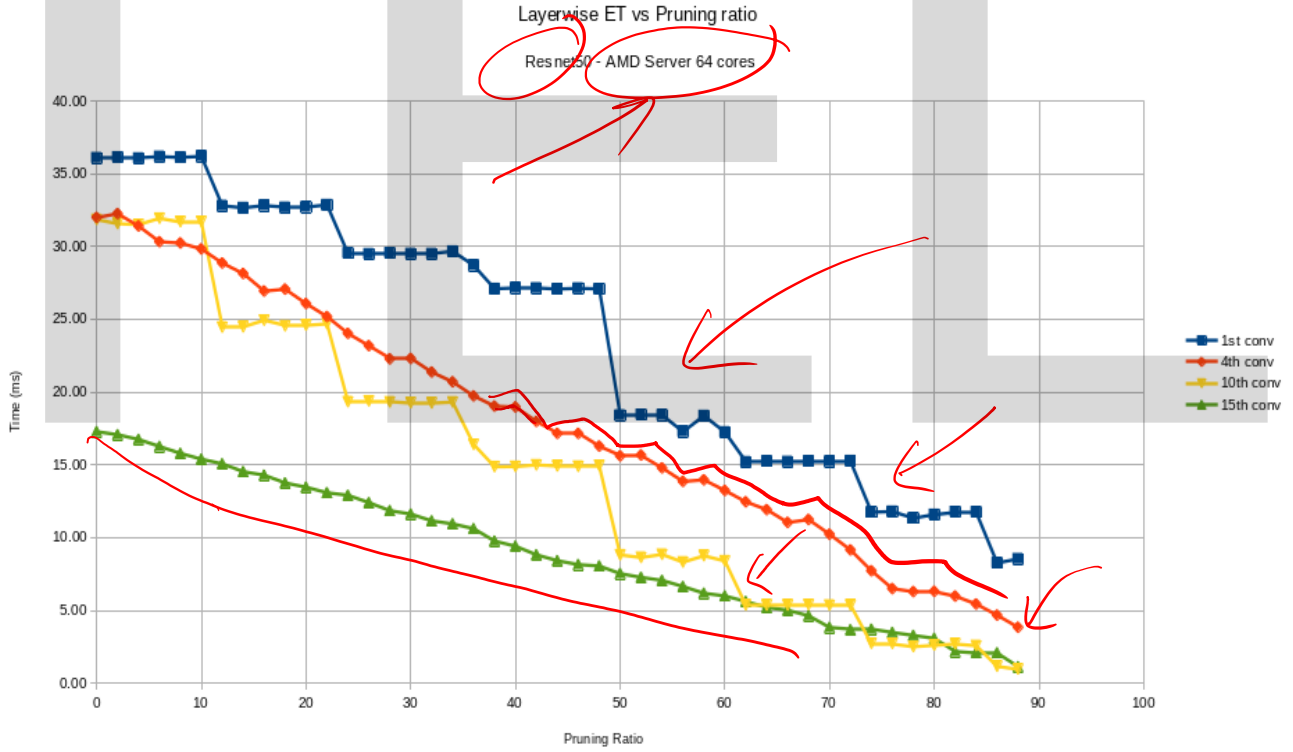
- Different layers differently to pruning in terms of loss in accuracy
- Should we choose only the layers with lower loss in accuracy to prune?
- But do all layers have the same execution time?
- Pruning some layer with low loss in accuracy may fail to yield much reduction in total execution time
- We need to strike a balance in terms of execution time saved and the loss in accuracy incurred



NPTEL

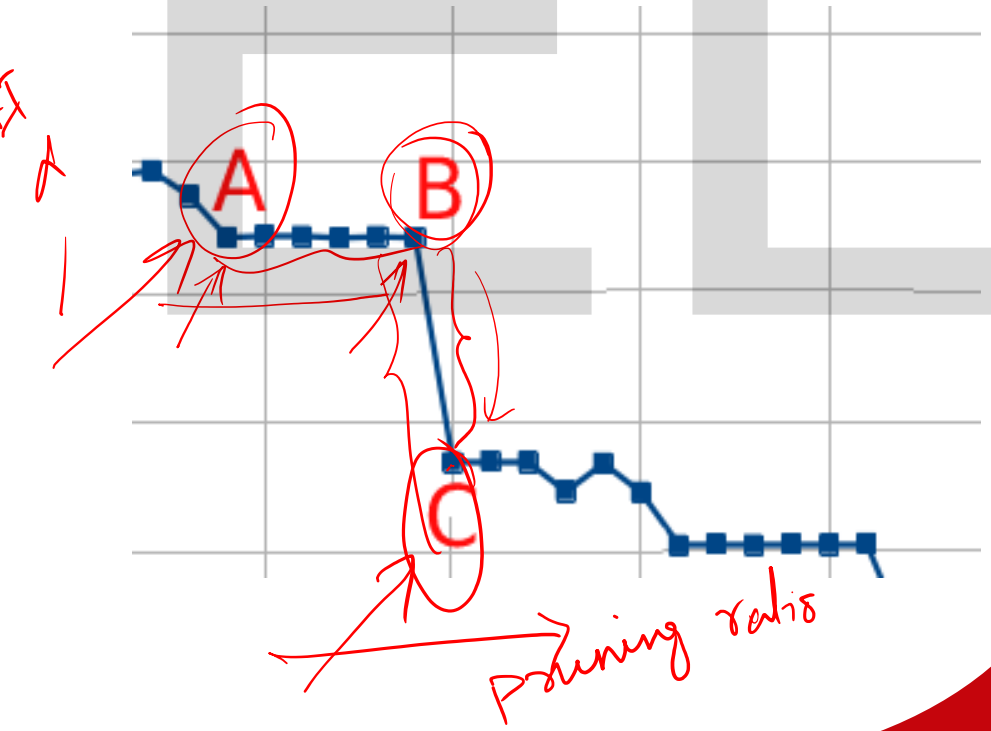
How do Execution Time Change with Pruning?

- As we prune a layer, we are reducing the number of parameters, this should result in a gradual decrease in its Execution time.
- But is this decrease in Execution time linear?
- It is observed that even for uniformly increasing pruning ratio, the execution time does not decrease linearly but in a staircase pattern



Platform Awareness in Pruning

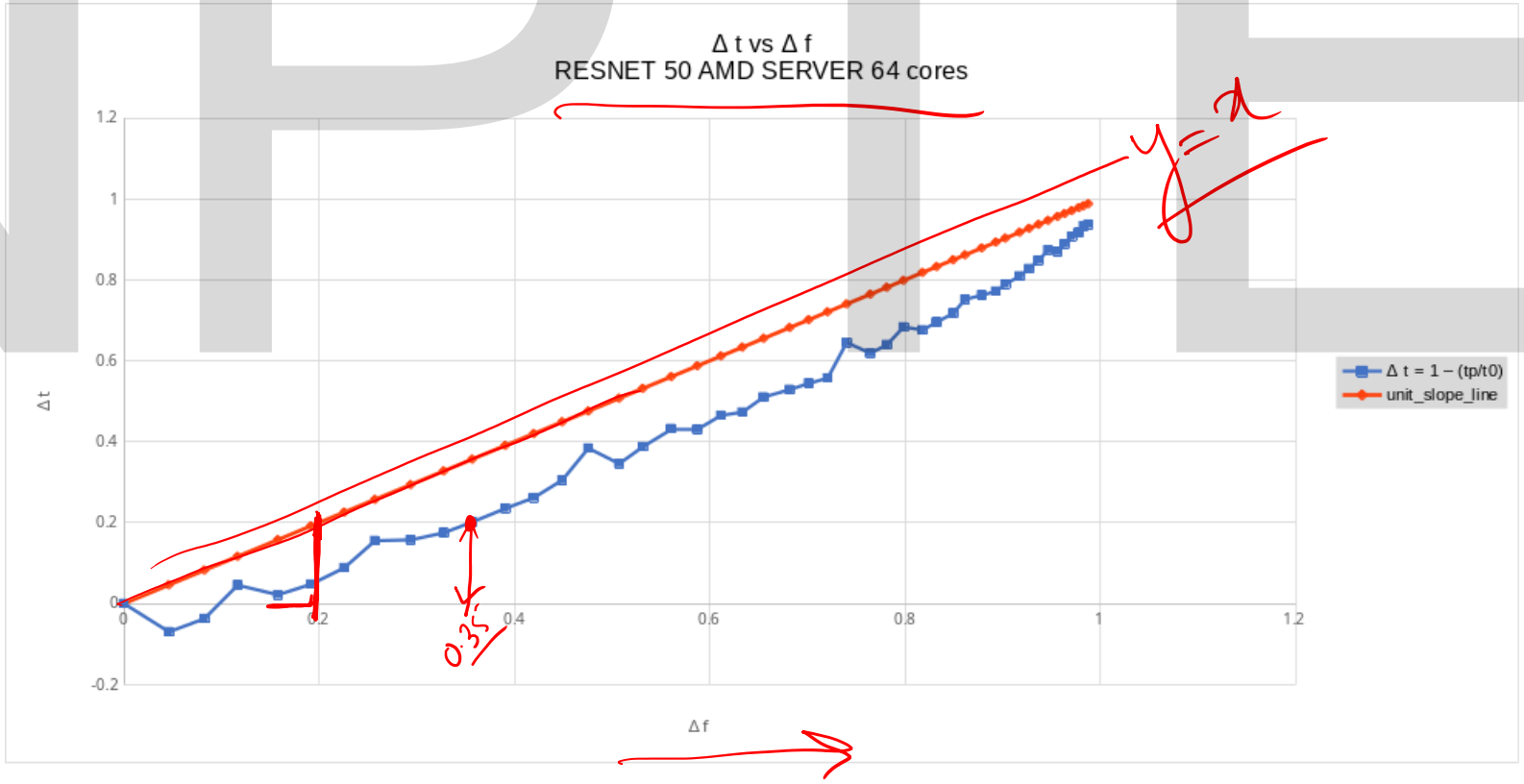
- It is observed a non-linear decrease in execution time as the layers are pruned
- The exact trend may differ from platform to platform of execution
- In the case of staircase pattern in decreasing of execution time, we want the model to take benefit of the sharp decrease in execution time
- To get the benefits from pruning, we need to prune at or nearby the drop points
- Pruning at the flat regions won't translate to a reduction in execution time but may result in loss of accuracy
- Pruned model at point A or C is desirable whereas pruned model at point between A and B is not



NPTEL

Platform Awareness in Pruning

- It is seen that FLOPS needs to be reduced comparatively more to gain similar benefit in execution time
- There are also irregularities in the behaviour speed-up trend that we may benefit from



Δt represents the normalised speedup

Δf represents the normalised reduction in FLOPS

NPTEL

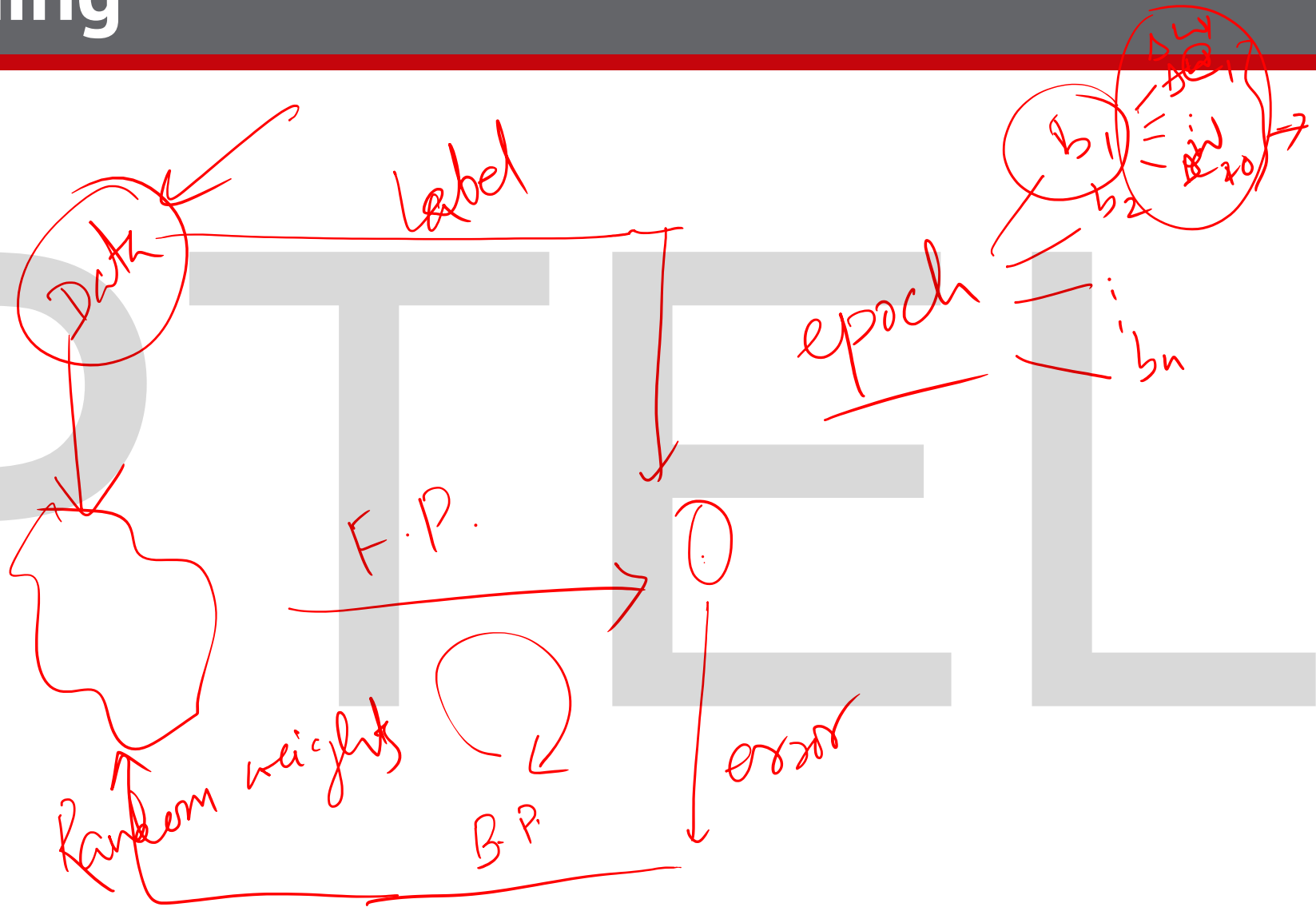
Agenda

- Recap
- DNN Pruning (contd.)
- **Parallelism in DNN Training**

NPTEL

Distributed Training

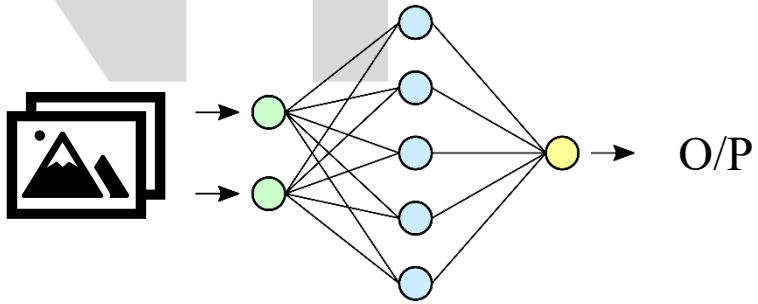
- Data Parallelism
- Model Parallelism
- Pipeline Parallelism
- Tensor Parallelism

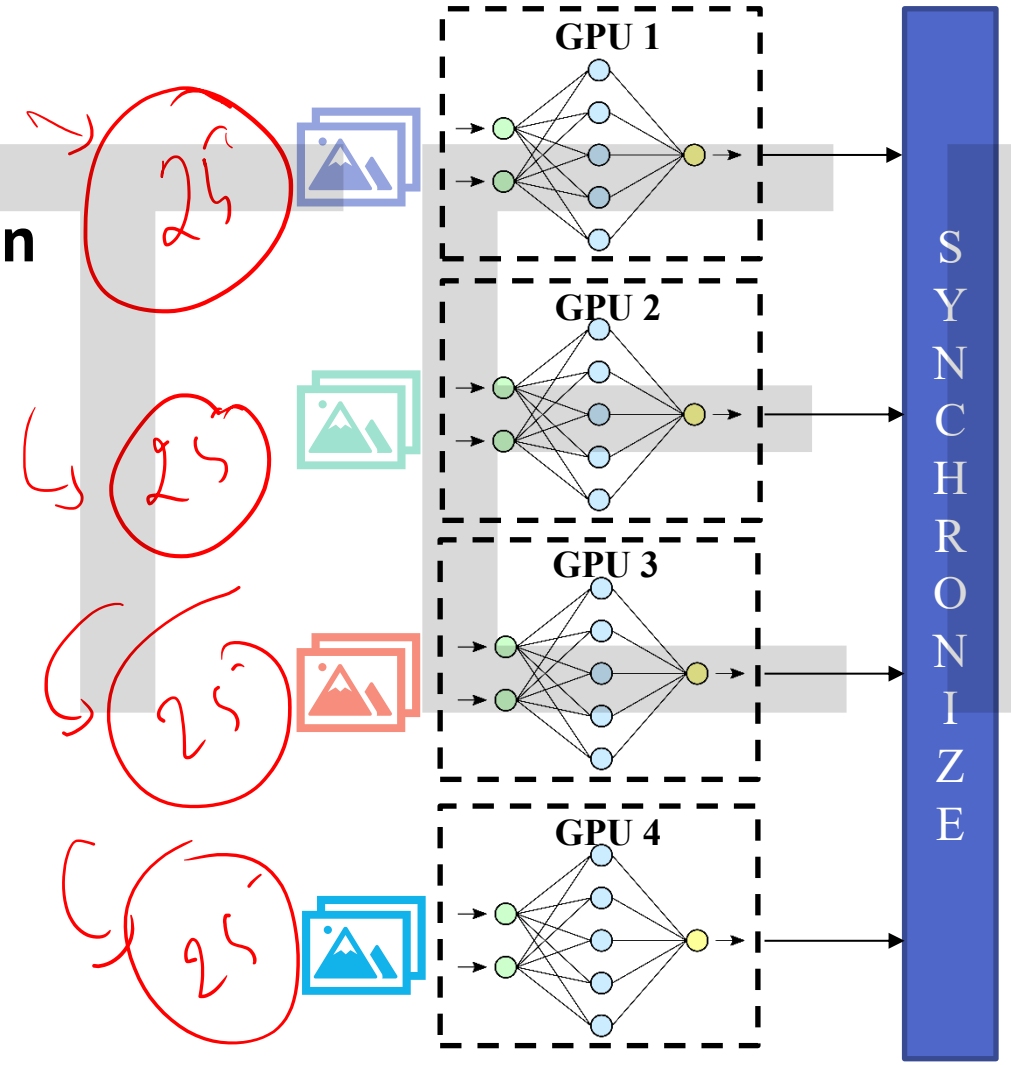


NPTEL

Data Parallel Training

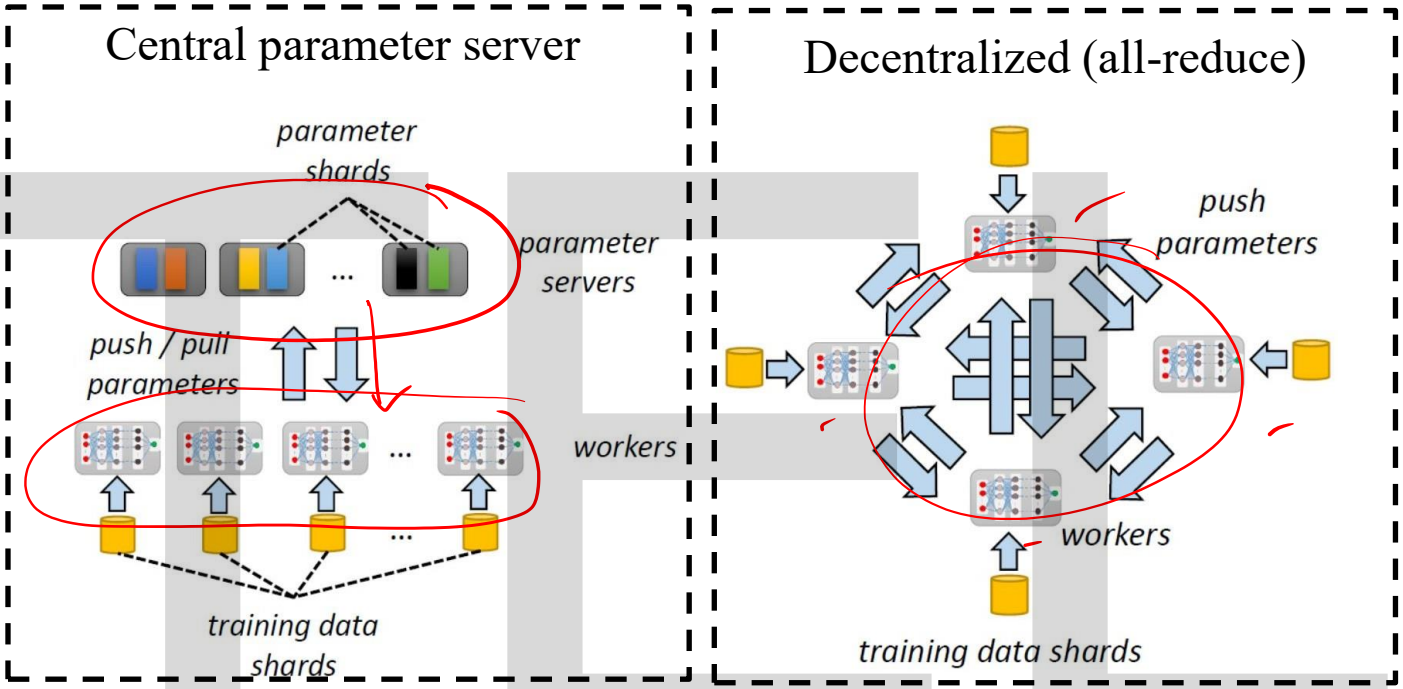
- Replicate the model across all GPUs
- Shard (break) the inputs
- Forward and backward propagation on same model but with different data
- Average parameters (weights/bias), updates (gradients) at every iteration





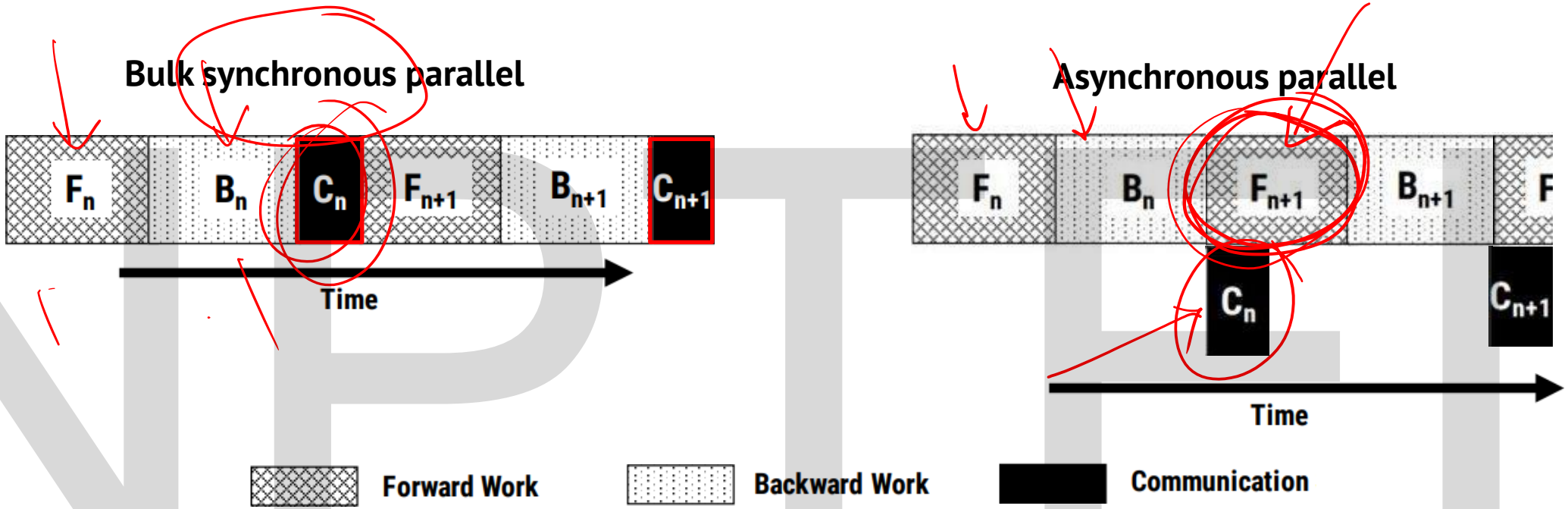
Parameter Synchronization

- **How to synchronize?**
  - Centralized vs decentralized
- **When to synchronize?**
  - After every batch of after a few?
- **How to reduce the overhead of synchronization?**



Figures source: Mayer, Ruben; Jacobsen, Hans-Arno. Scalable deep learning on distributed infrastructures: Challenges, techniques, and tools. ACM Computing Surveys (CSUR), 2020, 53.1: 1-37.

When to Synchronize?



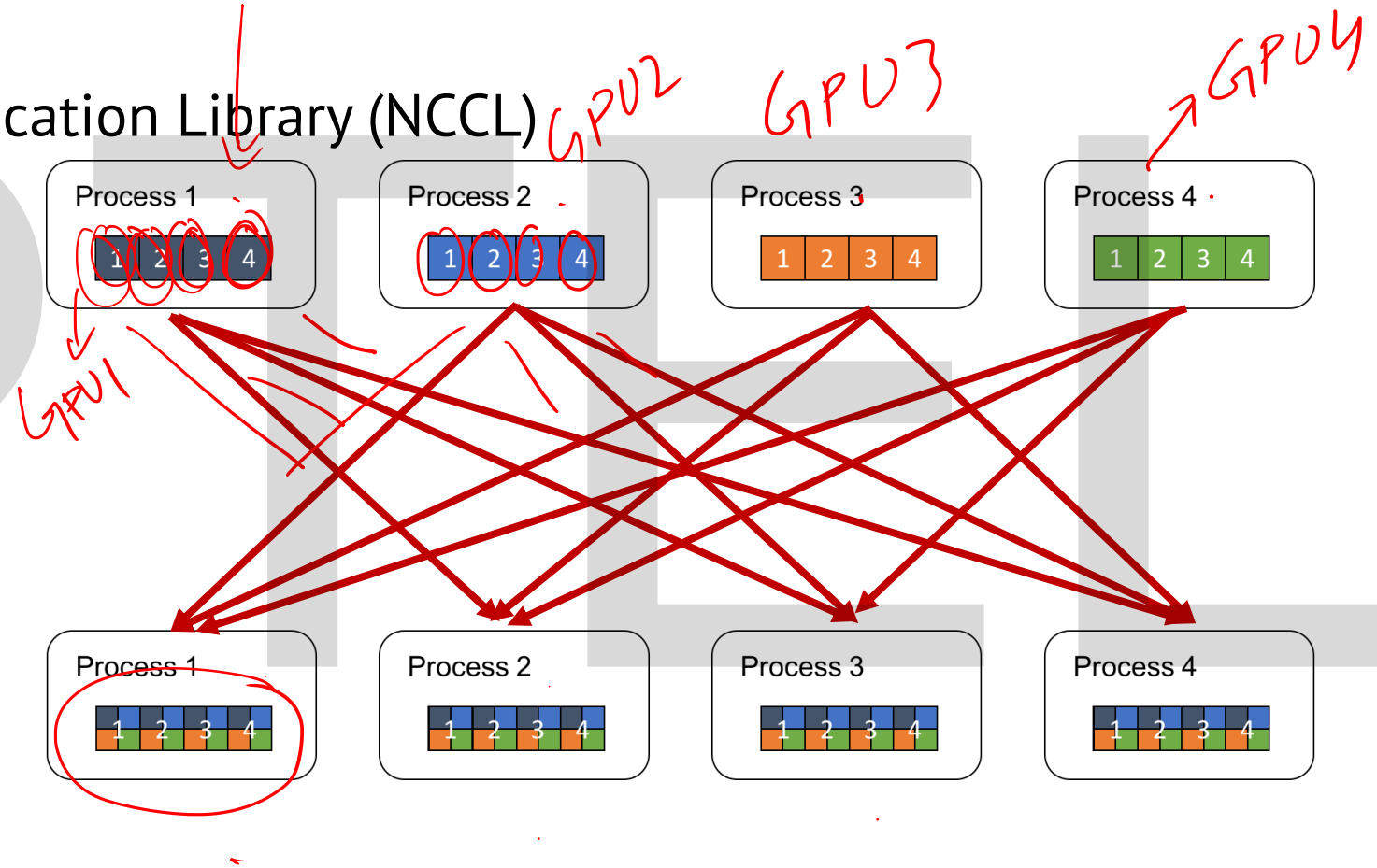
- More communication overhead
- Prone to stragglers (falling behind)
- No staleness

- Less communication overhead
- Staleness
- E.g., gradient accumulation

NPTEL

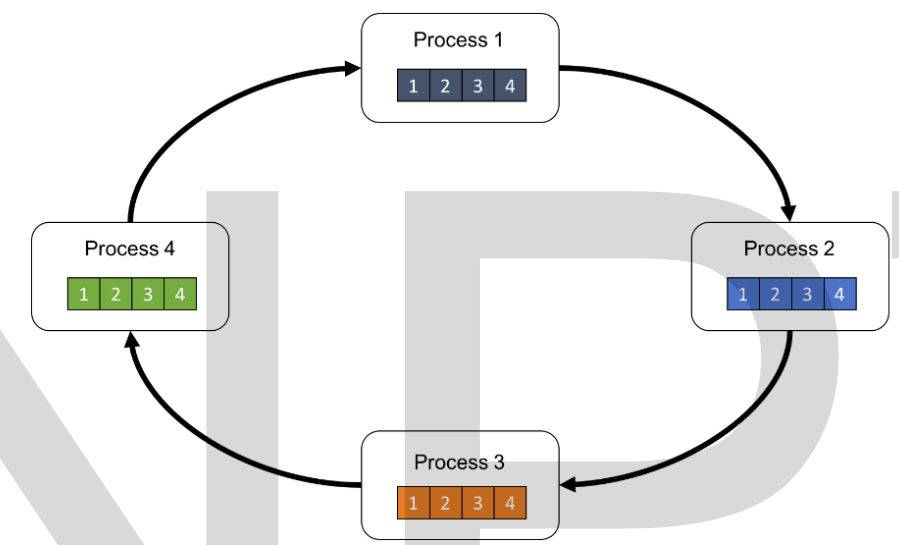
Reducing Synchronization Overhead

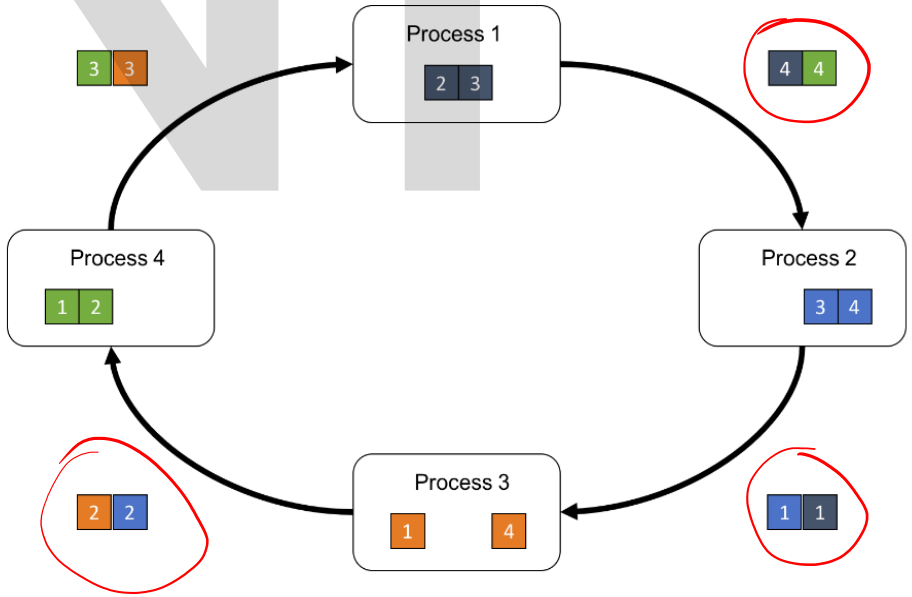
- **NCCL All Reduce**
  - NVIDIA Collective Communication Library (NCCL)
- **Simple strategy**
  - All-to-all communication

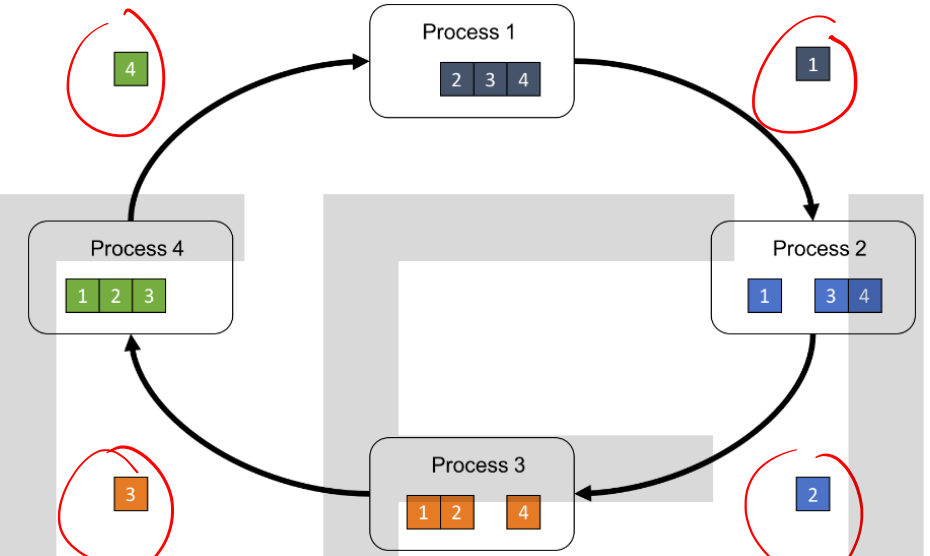


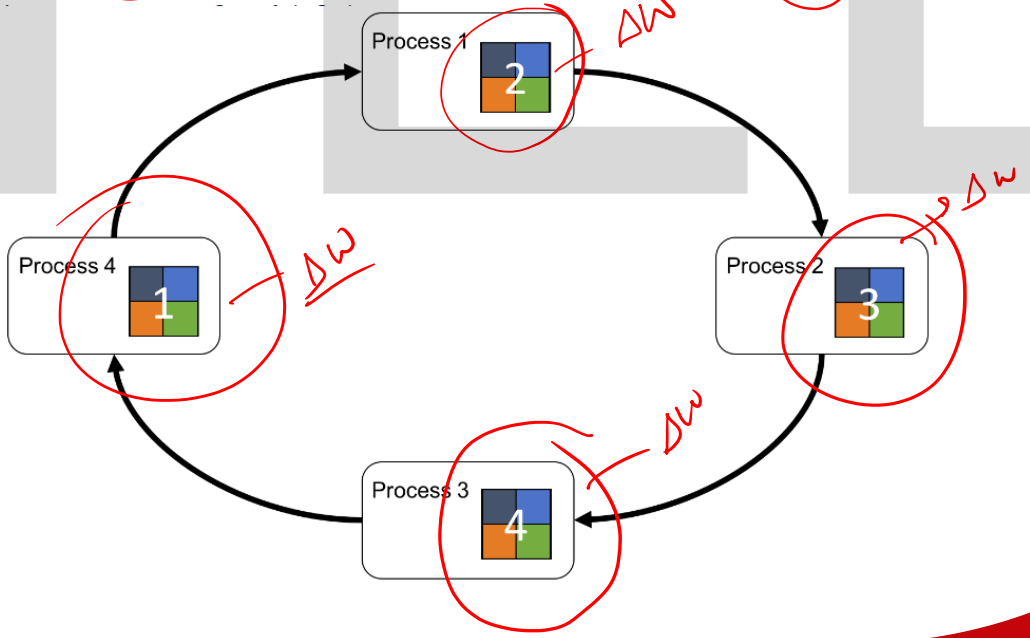
What is the amount of the all to all communication if there are P processes?

Ring-AllReduce



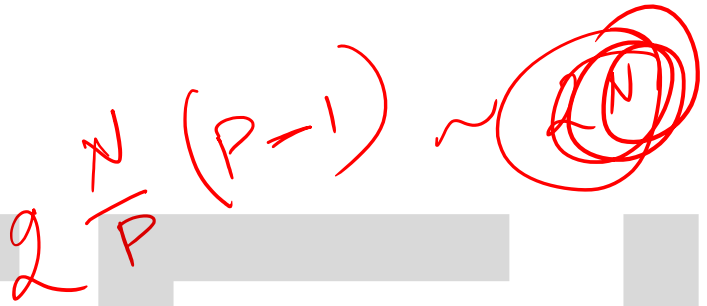






AllReduce Communication

- **AllReduce**
  - Number of communications: P*(P-1)
- **Ring allReduce**
  - Each process sends an array, the size of which is N/P, P-1 times
  - Next, each process again sends an array of the same size P-1 times
  - The total amount of data each process sends throughout the algorithm is 2N(P-1)/P, which is practically independent of P



Essentially, communication is independent of number of workers. Then what limits scalability of data parallel training?

NPTEL



THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)